Akademia Górniczo-Hutnicza w Krakowie			
Faculty: AGH	Academic Year 2025/2026	Year of study	Fields of science:
Student(s) name:	Simon GANIER-LOMBARD & Daniil STEPANOV		
Course:	**Real Time Operating Systems in Control Applications**		
Exercise nr:	**RTOS-03**		
Exercise subject:	**Client-server architecture: distributed calculator**		
Lecturer:	dr inż. Grzegorz Hayduk		
Date: 22.04.2026			Grade:

1 Exercise purpose

The goal of this exercise is to learn the client-server model. We build a small calculator that runs on many processes.
One server takes requests from users and workers clients do the math and send results back.
This helps us understand how processes talk to each other in QNX.

2 Assumptions / Theory

The client-server model splits work between two kinds of processes:
- The client asks for a service (operator logic or user-end)
- The server gives the answer.

In QNX we use MsgSend, MsgReceive and MsgReply to pass messages. The server first calls name_attach to register a name. The clients use name_open to find the server by that name. We use the Reply-driven scheme so worker clients wait blocked until the server has a job for them.

3 Description of implementation

The system has three kinds of processes.
- The main Calc Server takes requests from end-user clients and sends them to the right worker client (add, sub, mul or div).
- The end-user client asks the user for an operation and two numbers, then sends them to the server.
- The worker client is registered at the server, waits for a job, does the math and sends the answer back.

All parts use QNX messages (MsgSend, MsgReceive, MsgReply) and the name server for finding the Calc Server by name. See the full commented code in the .zip file.

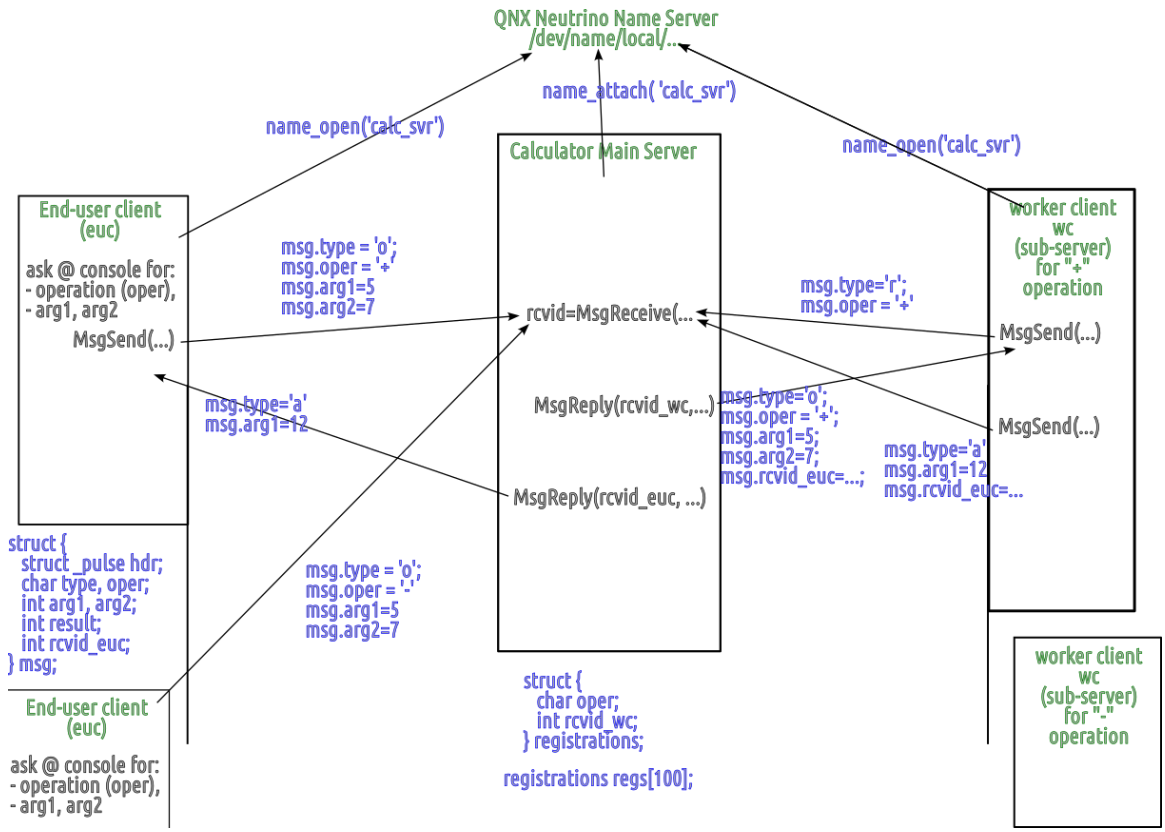

Figure 1. Architecture of the distributed calculator with message flow.

4 Conclusions

This exercise helped us understand how processes can work together in QNX. We learned how to use name_attach and name_open so the server and clients can find each other by name. Sending and receiving messages with MsgSend, MsgReceive and MsgReply is simple but you must plan the message format well.

The Reply-driven schema is useful because the worker clients stay ready and are only woken up when there is a real job. One hard part was to close / wipe the client operator if it's ailed, for this we learn how to use pulse to do what we want. When the server must reply to the correct end-user client after the worker finishes. The main advantage of this model is clean separation of tasks.